

Intelligence Artificielle

Résolution des Problèmes

(Recherche)

Un exemple introductif

Le problème

*On dispose de deux bidons: bidon4 et bidon3, de capacité respective 4 et 3 litres. **Aucun d'entre eux n'est gradué.** On dispose d'une pompe pour remplir les bidons. **Comment peut-on obtenir exactement 2 litres dans bidon4 ?***

Sa Représentation

1-Représentation graphique



Situation au début

Situation à la fin

Pas appropriée pour l'ordinateur
Contraintes non explicites

Plus appropriée

2-Représentation Symbolique

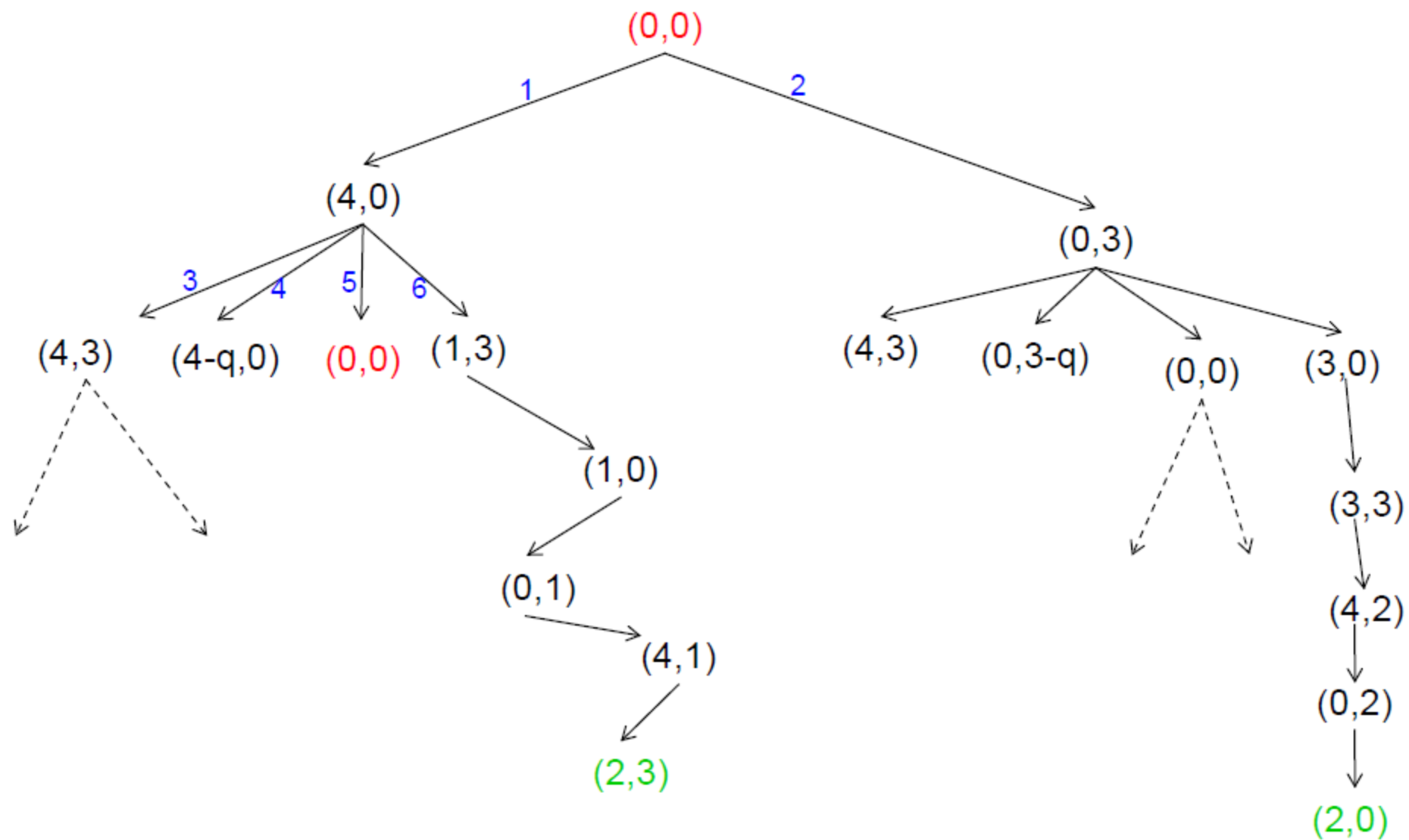
-Configurations possibles (V_4 , V_3) tel que:

- $V_4 \leftarrow 0, 1, 2, 3 \text{ ou } 4$

- $V_3 \leftarrow 0, 1, 2 \text{ ou } 3$

-Configuration initiale (0,0)

-Configuration finale (2,*)



Exemple : Puzzle-n

2	8	3
1	6	4
7		5



1	2	3
8		4
7	6	5

- états: **numéros des positions des plaquettes (ignorer les positions intermédiaires)**
- opérateurs: **déplacer la case vide à gauche (L), à droite (R), en haut (U), en bas (D)**
- test: **état courant = état final**
- fonction-coût: **chaque déplacement de la case vide vaut 1, coût total = nombre total de déplacements de la case vide**

remarque: **solution optimale pour puzzle-n est NP-difficile**

Haut

2	8	3
1	6	4
7		5

Haut

2	8	3
1		4
7	6	5

Gauche

2		3
1	8	4
7	6	5

Bas

	2	3
1	8	4
7	6	5

Droite

1	2	3
	8	4
7	6	5

1	2	3
8		4
7	6	5

dimension de l'espace d'états (machine à 10 millions états/sec)

puzzle-8 181,440 → 0.18 sec

puzzle-16 $.65 \times 10^{12}$ → 6 jours

puzzle-24 $.5 \times 10^{25}$ → 12 milliards d'années

Approches de Recherche

- Recherche non informée : exploration aveugle de l'espace des solutions (ex: recherche en profondeur, recherche en largeur)
- Recherche informée : utilisation de l'information heuristique pour guider la recherche vers la solution (ex: recherche heuristique, recherche A*)

Introduction aux algorithmes de recherche

Définir un problème de recherche

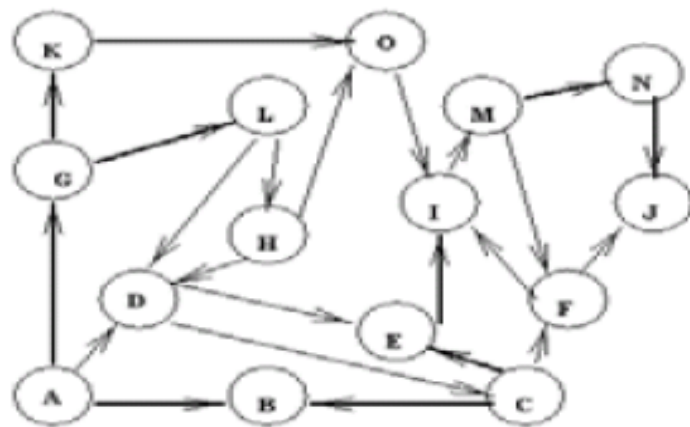
- Espace d'états
 - chaque état est une représentation **abstraite** de l'environnement
 - l'espace d'états est discret
- État initial
 - habituellement l'état de départ
 - parfois un ou plusieurs états hypothétiques
- Fonction "successeur " (Opérateurs)
 - fonction : [état $\rightarrow$ sous-ensemble d'états]
 - une représentation **abstraite** des actions possibles
- Test-solution
 - habituellement **une condition à satisfaire**
 - parfois la description explicite d'un état
- Coût du chemin
 - fonction : [chemin $\rightarrow$ nombre positif]
 - habituellement: coût du chemin = somme des coûts de ses étapes

Représentation: Graphes, Arbres

Etats → représentés par des **nœuds** (enregistrement)

Actions → représentées par des **arcs** (opération)

Graphe en général



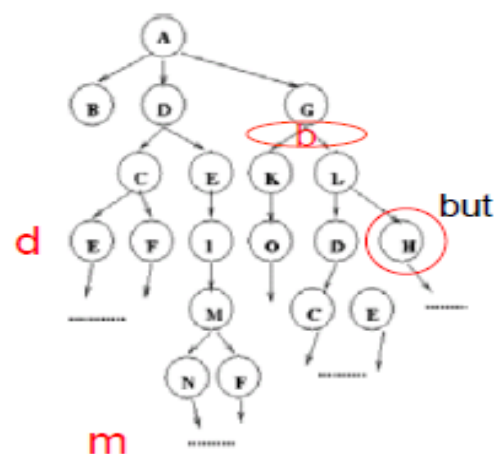
arbre utilisé



Méthodes: Critères d'évaluation

Les différentes méthodes de recherche sont évaluées selon les critères suivants:

- **Complétude:** est-ce que la méthode garantit de trouver une solution si elle existe?
- **Optimalité:** est-ce que la méthode trouve la meilleure solution s'il en existe plusieurs?
- **Complexité en temps:** combien de noeuds faut-il produire pour trouver la solution?
- **Complexité en espace:** nombre maximum de noeuds à conserver en mémoire pour trouver la solution?
- Les complexités en temps et en espace sont mesurées en fonction de:
 - ***b*** = facteur de branchement de l'arbre de recherche (nombre maximum de successeurs pour un état)
 - ***d*** = profondeur à laquelle se trouve le (meilleur) noeud-solution
 - ***m*** = profondeur maximum de l'espace de recherche (peut être ∞)



Concepts de base pour la recherche

En résumé un algorithme de recherche repose sur les éléments suivants:

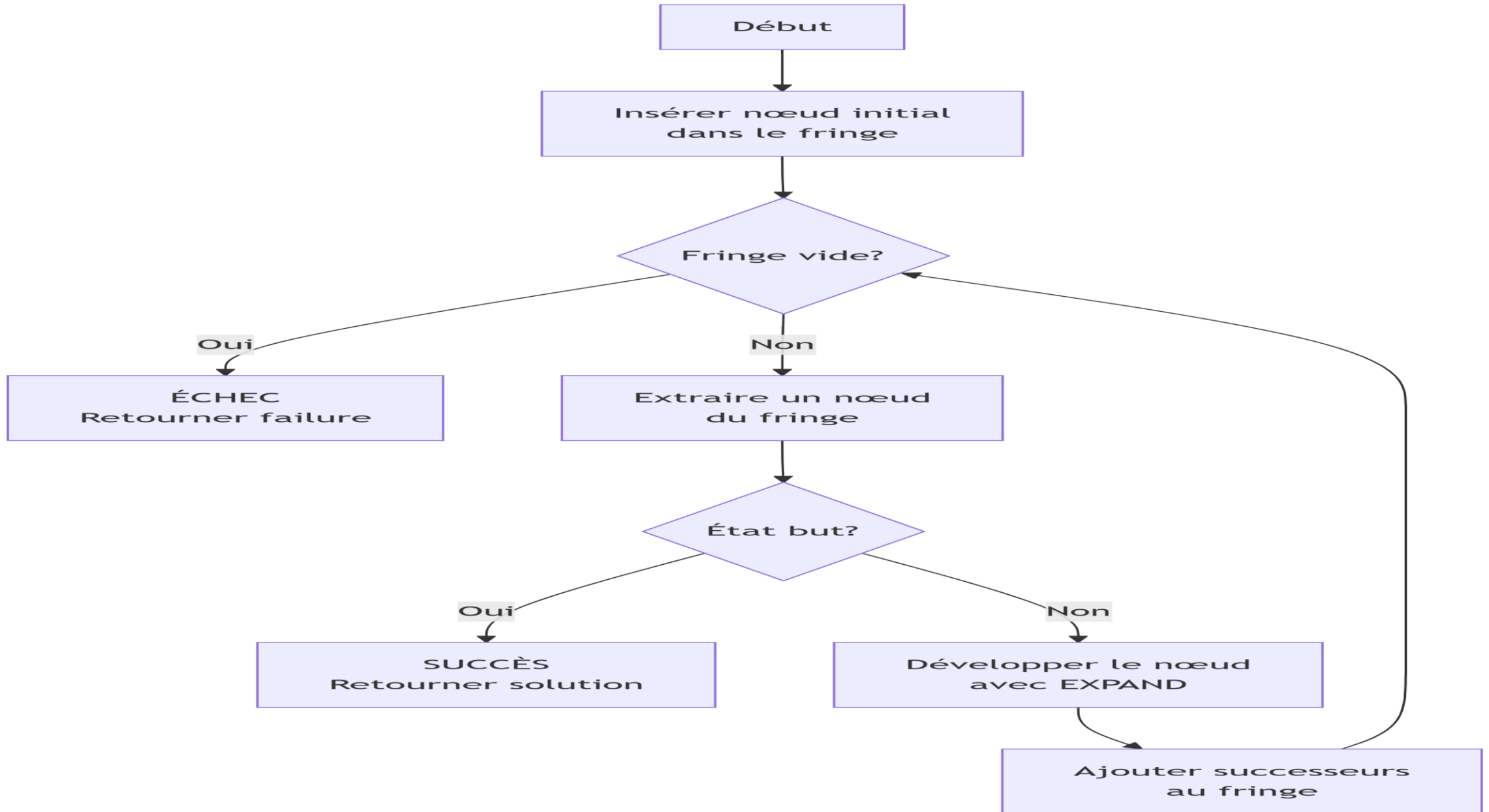
- **Arbre de recherche**
- **Recherche d'un nœud (état + pointeurs chemin)**
- **Expansion (développement) d'un nœud (recherche des successeurs)**
- **Stratégie de recherche (décision sur les nœuds à développer):**
à chaque étape du processus de recherche,
déterminer quel est le noeud à "étendre" (développer)

Algorithmme de recherche dans les arbres

```
function TREE-SEARCH(problem, fringe) returns a solution, or failure  
  fringe  $\leftarrow$  INSERT(MAKE-NODE(INITIAL-STATE[problem]), fringe)  
  loop do  
    if fringe is empty then return failure  
    node  $\leftarrow$  REMOVE-FRONT(fringe)  
    if GOAL-TEST[problem](STATE[node]) then return SOLUTION(node)  
    fringe  $\leftarrow$  INSERTALL(EXPAND(node, problem), fringe)
```

```
function EXPAND(node, problem) returns a set of nodes  
  successors  $\leftarrow$  the empty set  
  for each action, result in SUCCESSOR-FN[problem](STATE[node]) do  
    s  $\leftarrow$  a new NODE  
    PARENT-NODE[s]  $\leftarrow$  node; ACTION[s]  $\leftarrow$  action; STATE[s]  $\leftarrow$  result  
    PATH-COST[s]  $\leftarrow$  PATH-COST[node] + STEP-COST(node, action, s)  
    DEPTH[s]  $\leftarrow$  DEPTH[node] + 1  
    add s to successors  
  return successors
```

Algorithme de Recherche dans les Arbres : Organigramme Principe



- **Structure de données : Le Fringe**
- **File (FIFO)** → Recherche en largeur (BFS)
- **Pile (LIFO)** → Recherche en profondeur (DFS)
- **File de priorité** → Recherche coût uniforme /Greedy/ A*

*Stratégies de recherche non-informées
(Aveugles)*

- **Les stratégies de recherche non-informées utilisent seulement les informations disponibles dans le problème**
 - Recherche en *largeur d'abord* (Breadth-First Search BFS)
 - Recherche en *coût uniforme* (Uniform-cost Search)
 - Recherche en *profondeur d'abord* (Depth-first Search)
 - Recherche en *profondeur limitée* (Depth-Limited Search)
 - Recherche *itérative en profondeur* (Iterative deepening depth-first Search)

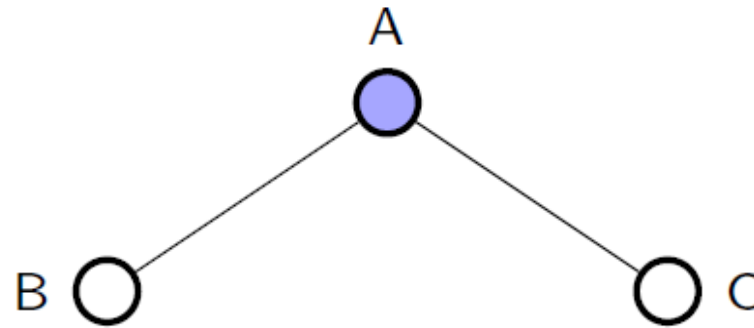
1- Recherche en largeur d'abord

- La fonction Insert-Fn ajoute les successeurs en fin de liste
- $fringe = [A]$

A
○

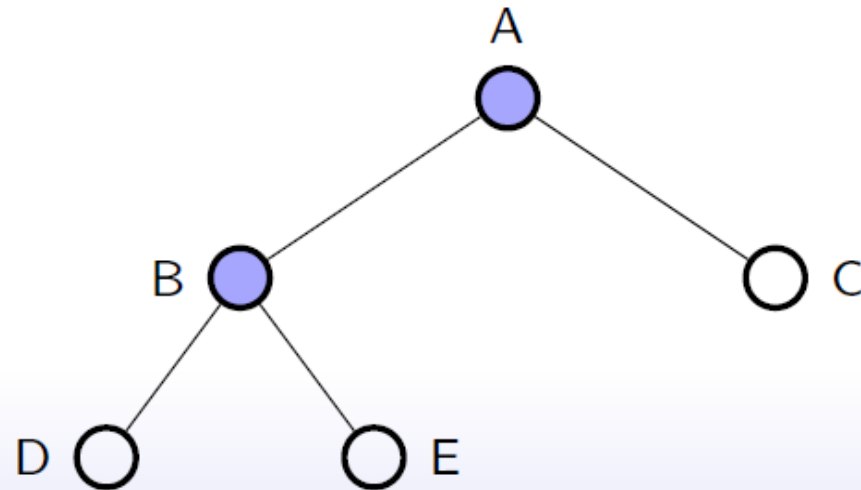
Recherche en largeur d'abord

- La fonction `Insert-Fn` ajoute les successeurs en fin de liste
- $fringe = [B, C]$



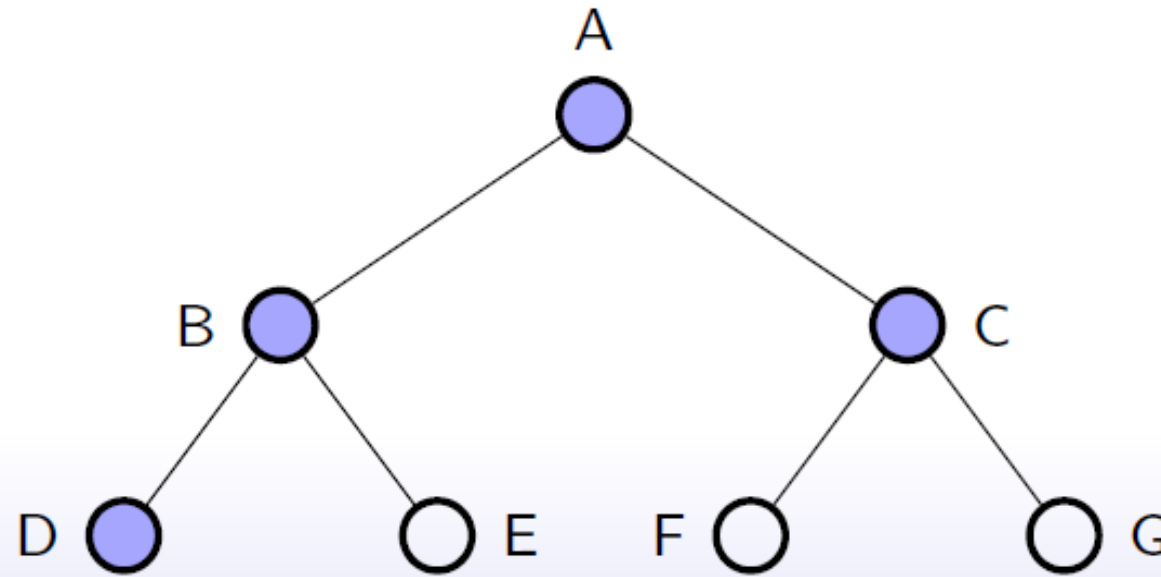
Recherche en largeur d'abord

- La fonction Insert-Fn ajoute les successeurs en fin de liste
- $fringe = [C, D, E]$



Recherche en largeur d'abord

- La fonction Insert-Fn ajoute les successeurs en fin de liste
- $fringe = [E, F, G]$



Recherche en largeur d'abord

- Complet, si b est fini
- Complexité en temps :

$$1 + b + b^2 + b^3 + \dots + b^d + (b^{d+1} - b) = O(b^{d+1})$$

- Complexité en espace : $O(b^{d+1})$ (garde tous les nœuds en mémoire)
- Optimale si coût = 1 pour chaque pas, mais non optimale dans le cas général

⇒ L'espace est le plus gros problème

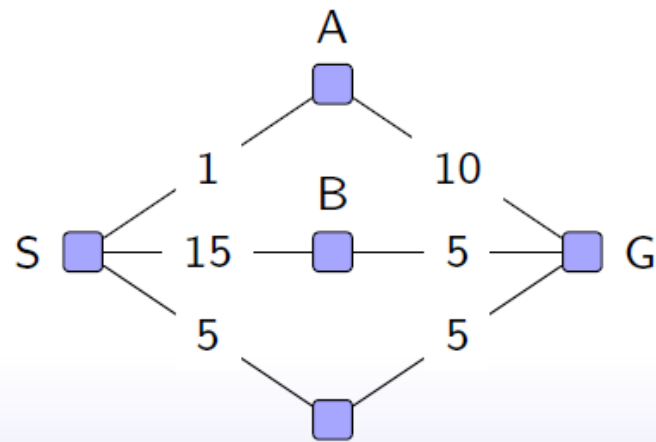
Recherche en largeur d'abord

- $b = 10$
- 10000 nœuds par seconde
- 1000 octets de mémoire pour un nœuds

Profondeur	Nœuds	Temps	Mémoire
8	10^9	31 heures	1 Teraoctet, 1024 Go
12	10^{13}	35 ans	10 Petaoctets, 1024 Teraoctet

2- Recherche en coût uniforme

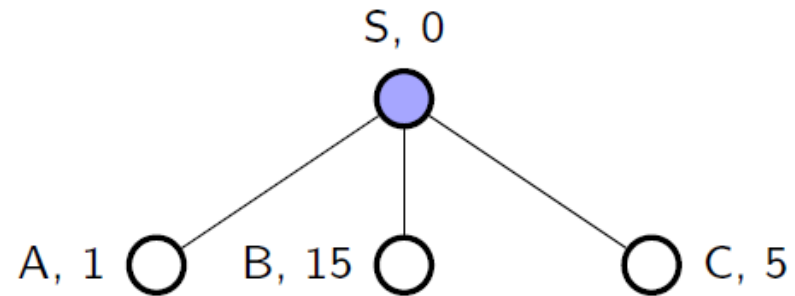
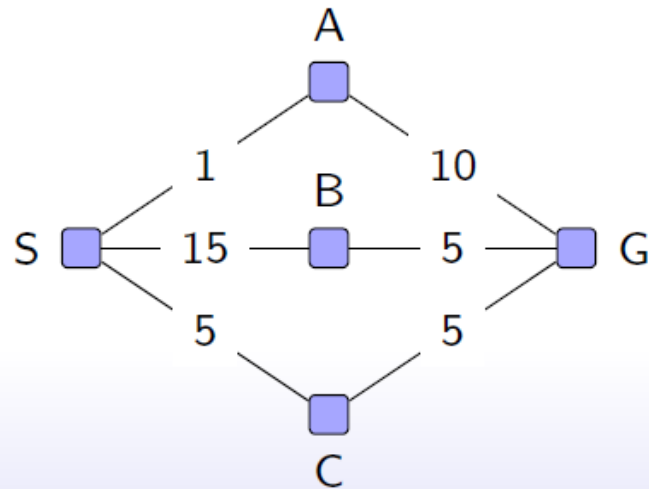
- La fonction Insert-Fn ajoute les nœuds dans l'ordre du coût $g(x)$
- $fringe = [(S, 0)]$



S, 0
○

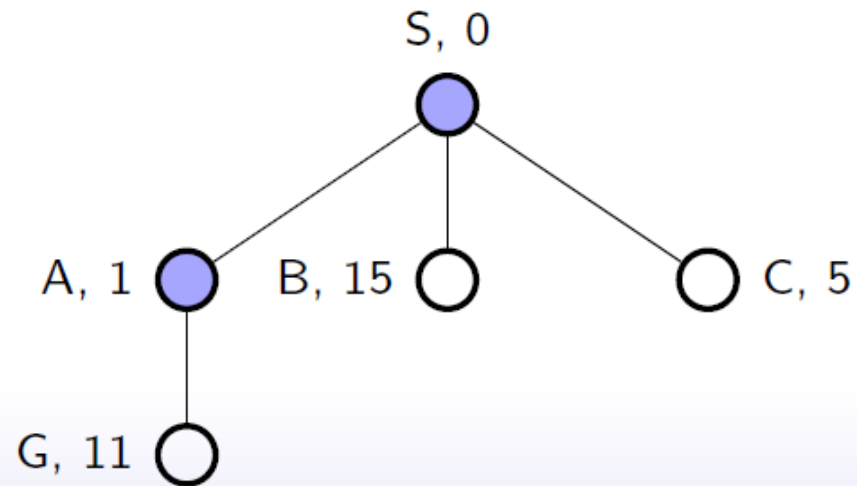
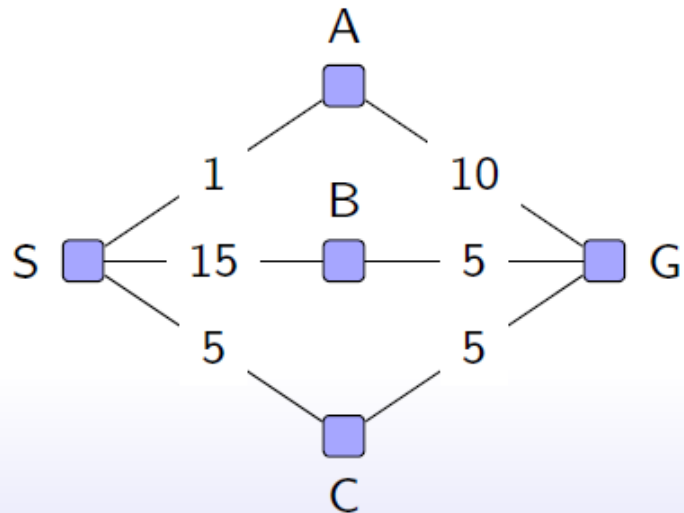
Recherche en coût uniforme

- La fonction Insert-Fn ajoute les nœuds dans l'ordre du coût $g(x)$
- $fringe = [(A, 1), (C, 5), (B, 15)]$



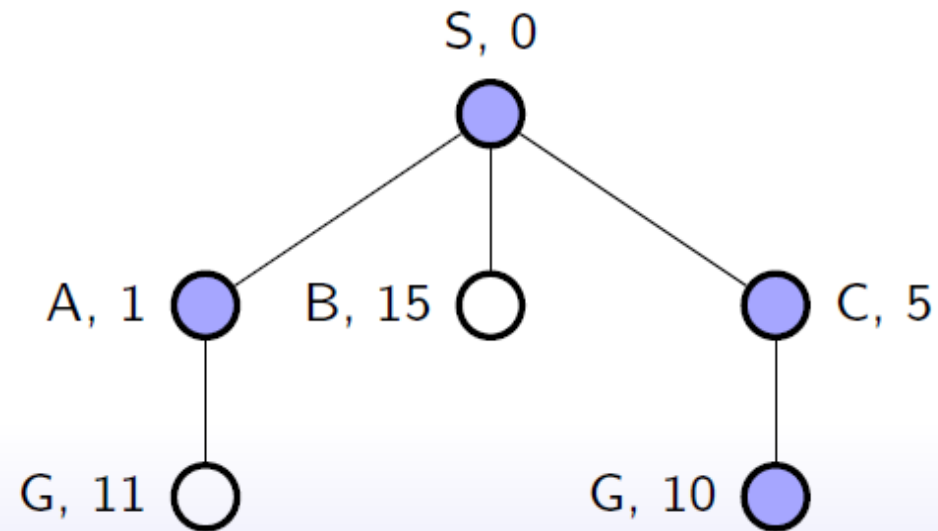
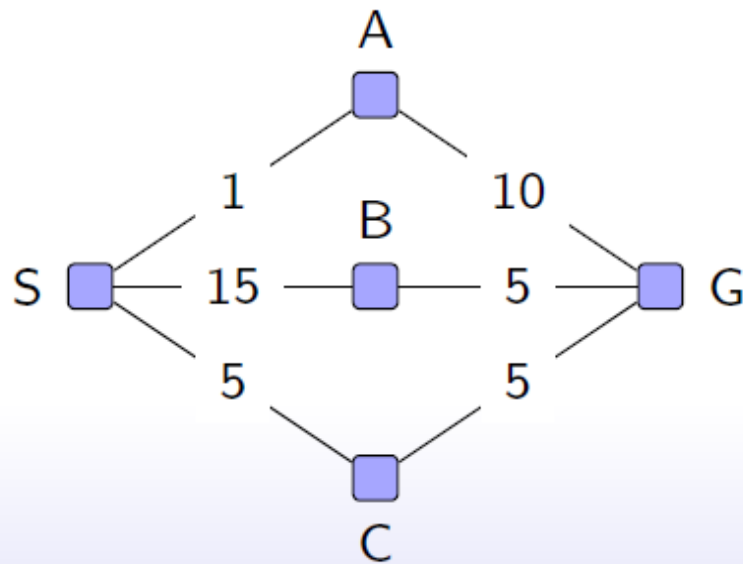
Recherche en coût uniforme

- La fonction Insert-Fn ajoute les nœuds dans l'ordre du coût $g(x)$
- $fringe = [(C, 5), (G, 11), (B, 15)]$



Recherche en coût uniforme

- La fonction Insert-Fn ajoute les nœuds dans l'ordre du coût $g(x)$
- $fringe = [(G, 11), (B, 15)]$

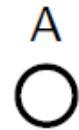


Recherche en coût uniforme

- Equivalent à la largeur d'abord si le coût est toujours le même
- Complet si le coût de chaque pas est strictement supérieur à 0
- Complexité en temps : nombre de nœuds pour lesquels $g \leq C^*$, où C^* est le coût de la solution optimale
- Complexité en espace : idem que la complexité en temps
- Optimale car les nœuds sont développés en fonction de g

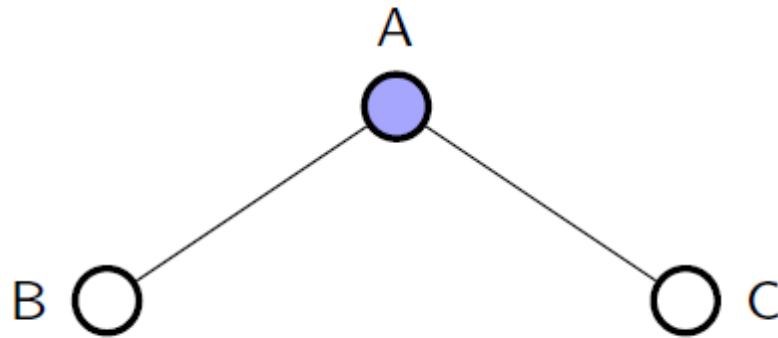
3- Recherche en profondeur d'abord

- La fonction Insert-Fn ajoute les successeurs en début de liste
- $fringe = [A]$



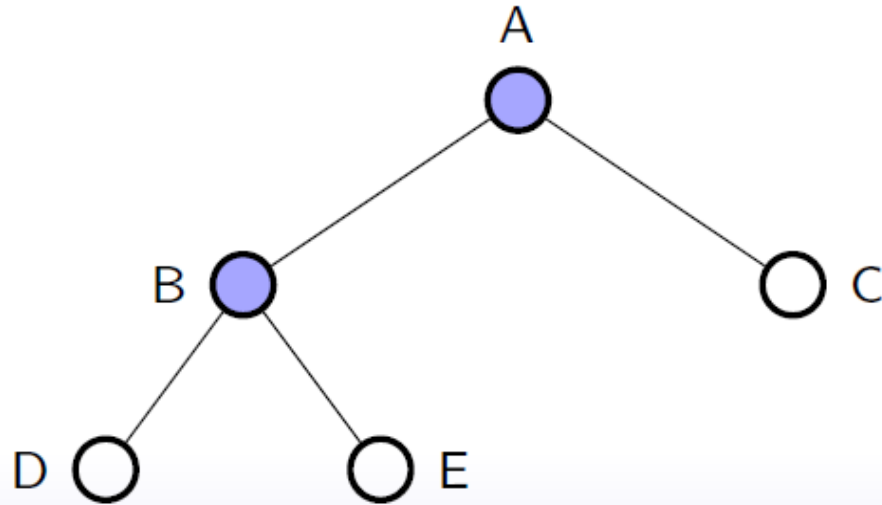
3- Recherche en profondeur d'abord

- La fonction Insert-Fn ajoute les successeurs en début de liste
- $fringe = [B, C]$



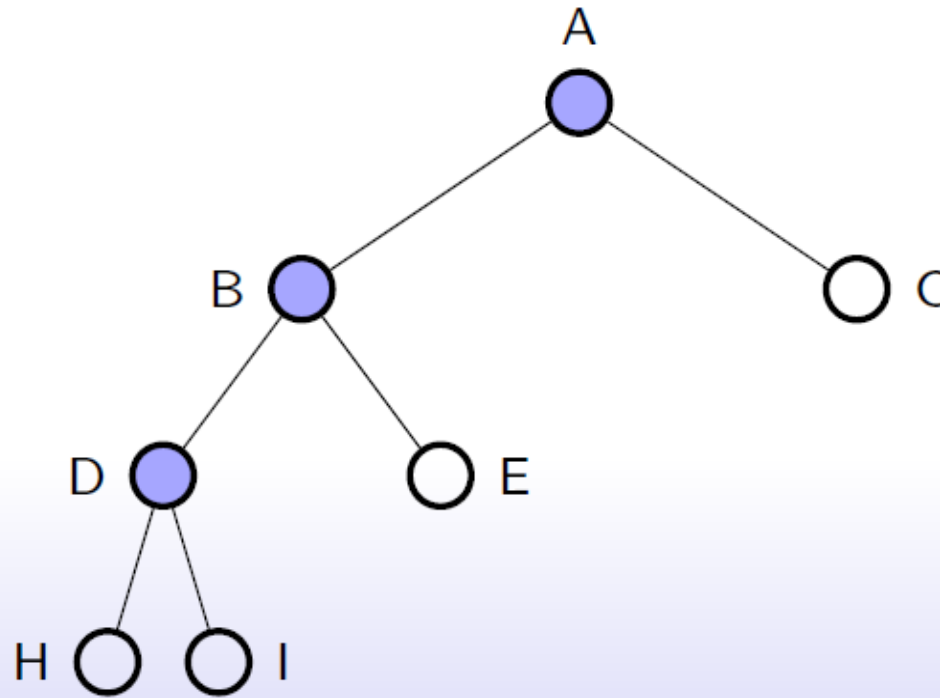
3- Recherche en profondeur d'abord

- La fonction Insert-Fn ajoute les successeurs en début de liste
- *fringe* = $[D, E, C]$



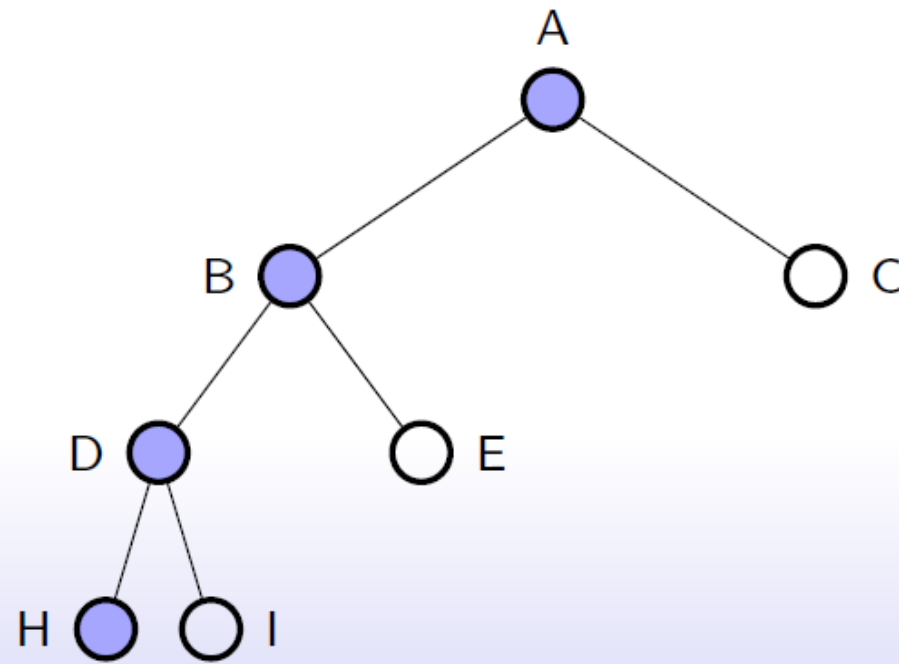
3- Recherche en profondeur d'abord

- La fonction Insert-Fn ajoute les successeurs en début de liste
- *fringe* = [H, I, E, C]



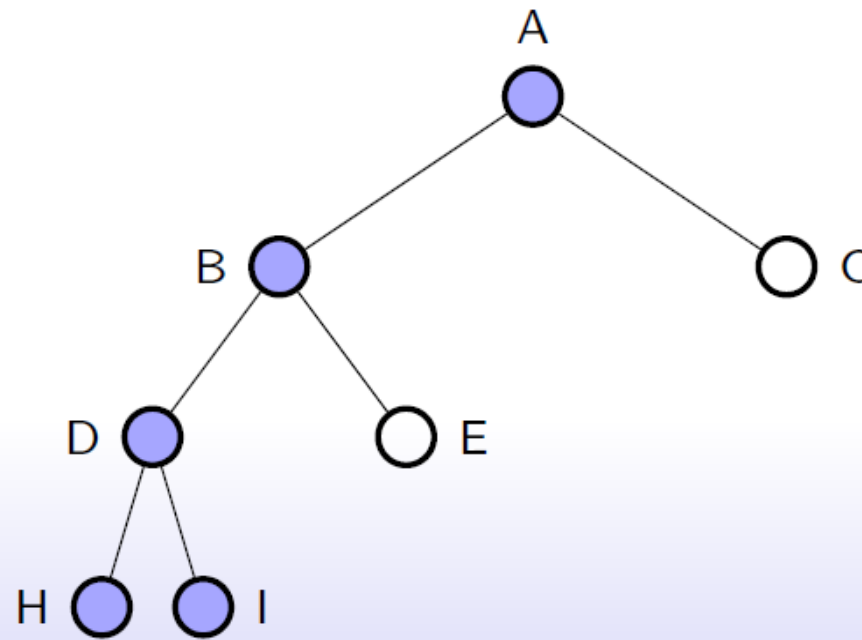
3- Recherche en profondeur d'abord

- La fonction Insert-Fn ajoute les successeurs en début de liste
- $fringe = [I, E, C]$



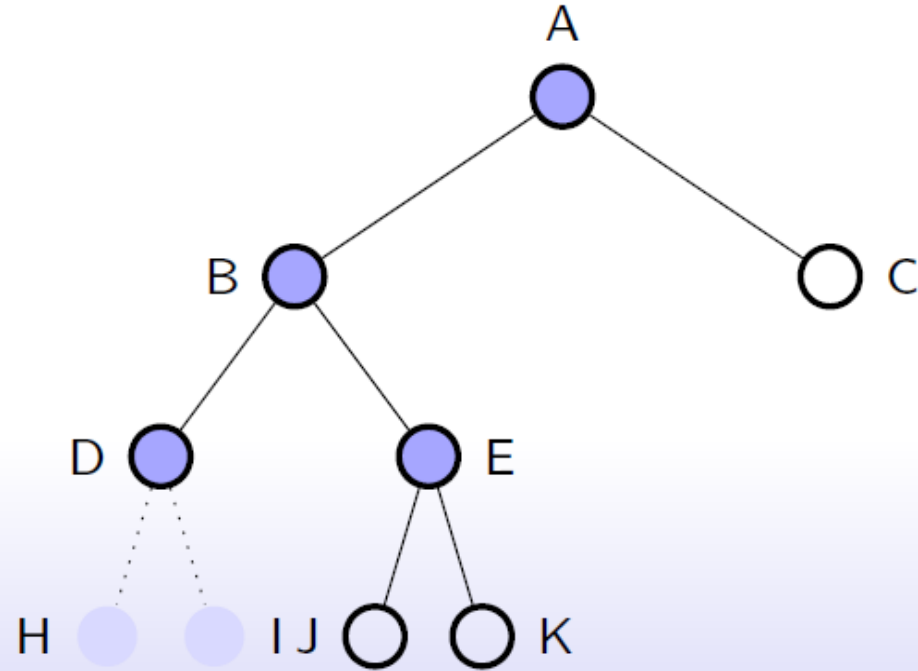
3- Recherche en profondeur d'abord

- La fonction Insert-Fn ajoute les successeurs en début de liste
- $fringe = [E, C]$



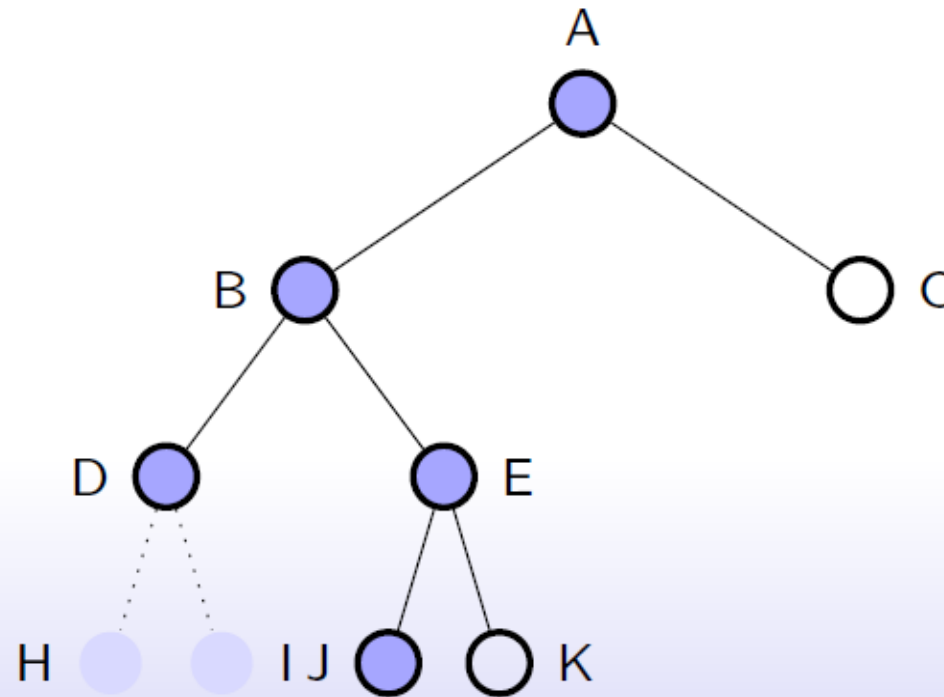
3- Recherche en profondeur d'abord

- La fonction Insert-Fn ajoute les successeurs en début de liste
- *fringe* = [J, K, C]



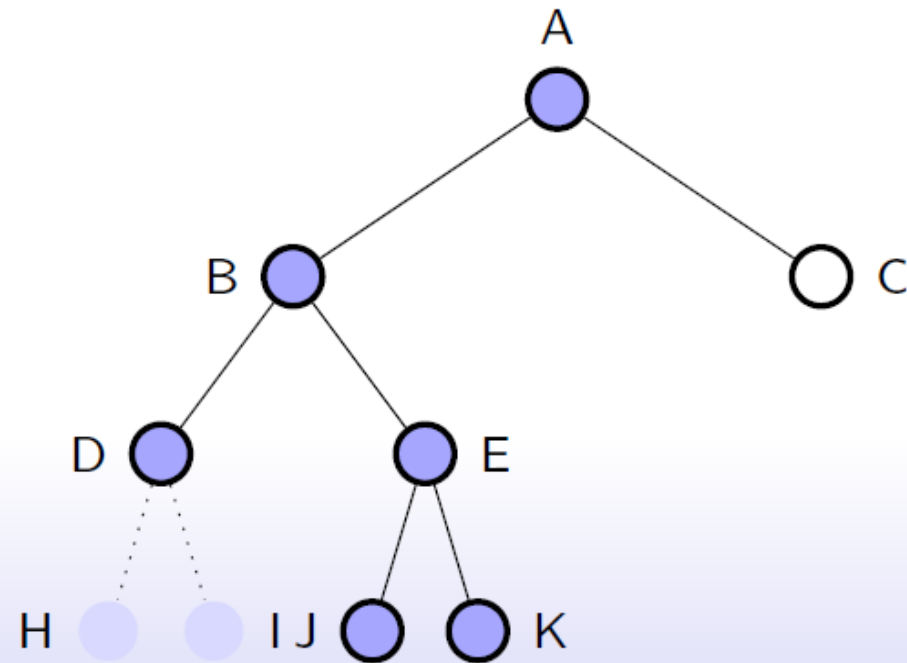
3- Recherche en profondeur d'abord

- La fonction Insert-Fn ajoute les successeurs en début de liste
- $fringe = [K, C]$



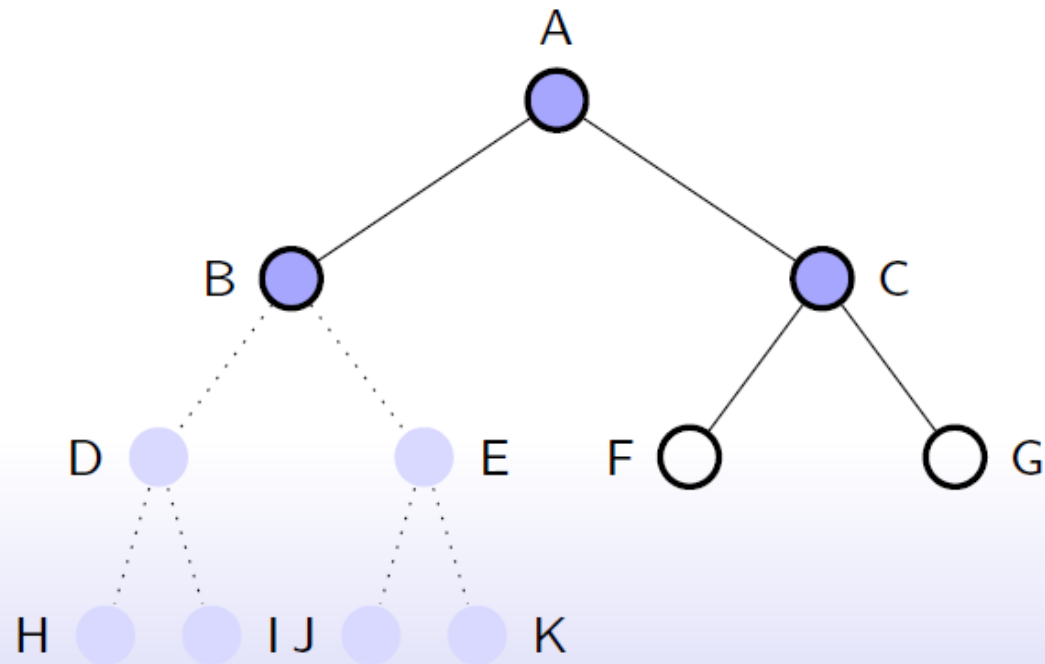
3- Recherche en profondeur d'abord

- La fonction Insert-Fn ajoute les successeurs en début de liste
- $fringe = [C]$



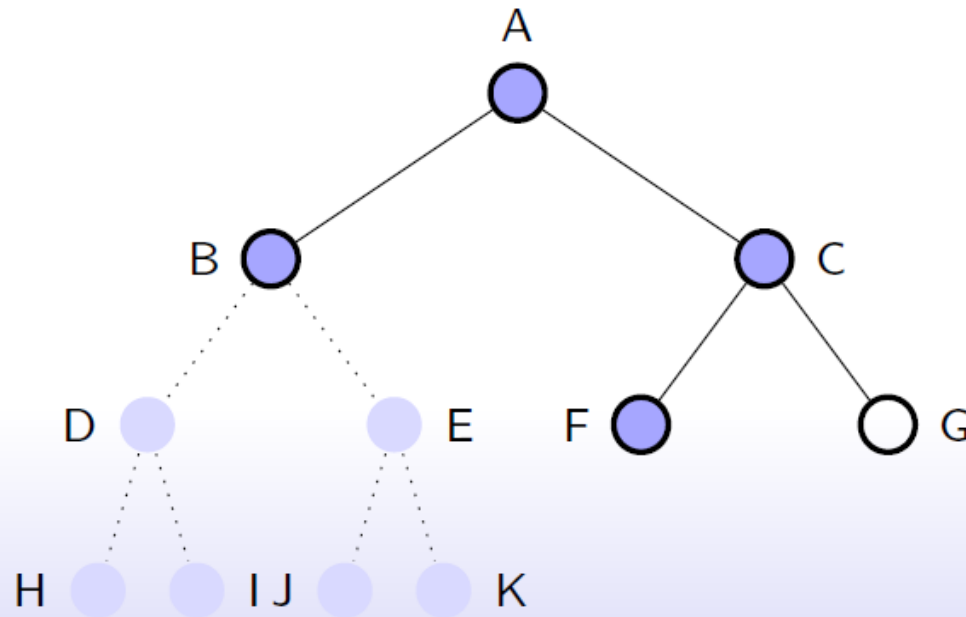
3- Recherche en profondeur d'abord

- La fonction Insert-Fn ajoute les successeurs en début de liste
- $fringe = [F, G]$



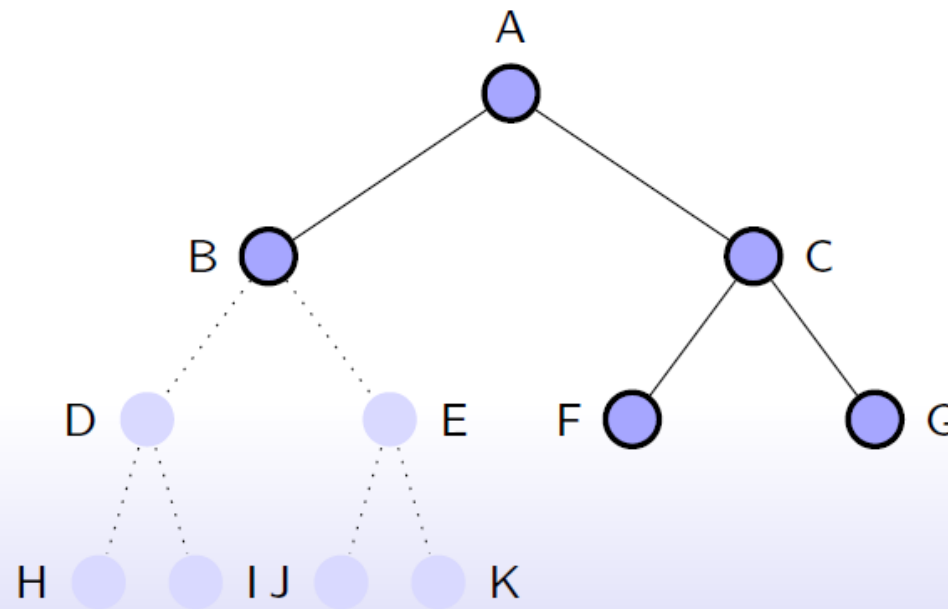
3- Recherche en profondeur d'abord

- La fonction Insert-Fn ajoute les successeurs en début de liste
- $fringe = [G]$



3- Recherche en profondeur d'abord

- La fonction Insert-Fn ajoute les successeurs en début de liste
- *fringe* = []

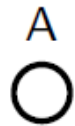


3- Recherche en profondeur d'abord

- Non complet dans les espaces d'états infinis ou avec boucle
 - Il est possible d'ajouter un test pour détecter les répétitions
- Complexité en temps : $O(b^m)$
 - Très mauvais si m est beaucoup plus grand que b
- Complexité en espace : $O(bm)$
 - Linéaire!
- Non optimale

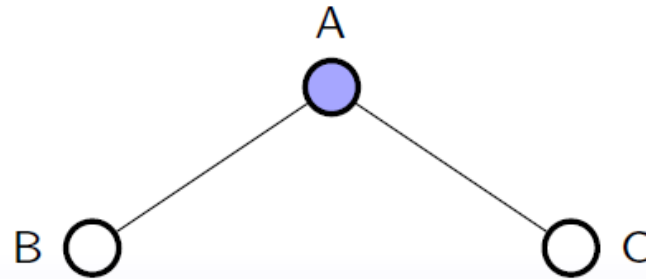
4- Recherche en profondeur limitée

- Algorithme de recherche en profondeur d'abord, mais avec une limite l sur la profondeur
 - Les nœuds de profondeur l n'ont pas de successeurs
- Exemple pour $l = 2$



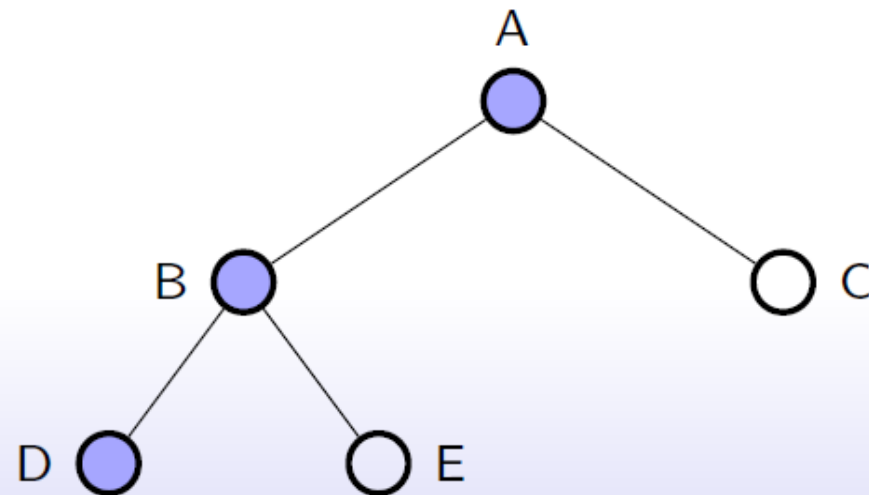
4- Recherche en profondeur limitée

- Algorithme de recherche en profondeur d'abord, mais avec une limite l sur la profondeur
 - Les nœuds de profondeur l n'ont pas de successeurs
- Exemple pour $l = 2$



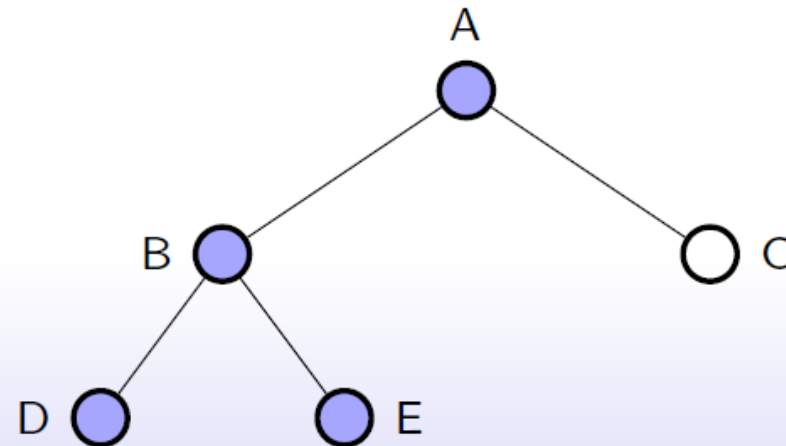
4- Recherche en profondeur limitée

- Algorithme de recherche en profondeur d'abord, mais avec une limite l sur la profondeur
 - Les nœuds de profondeur l n'ont pas de successeurs
- Exemple pour $l = 2$



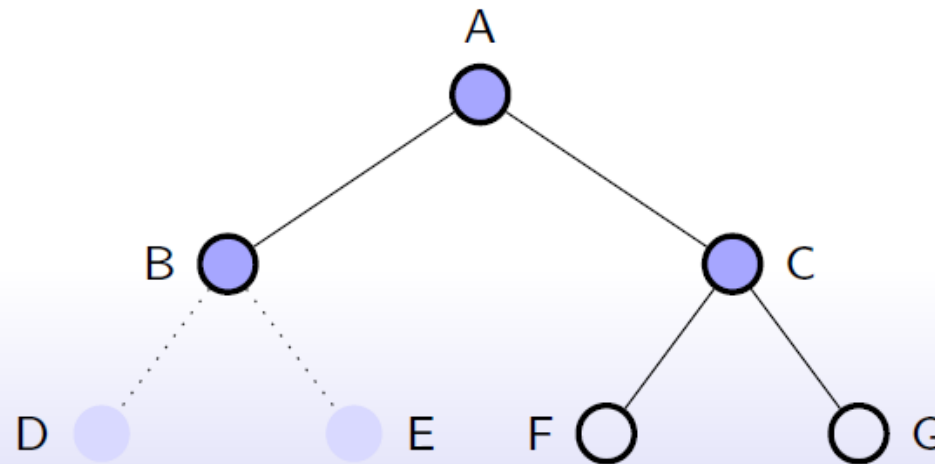
4- Recherche en profondeur limitée

- Algorithme de recherche en profondeur d'abord, mais avec une limite l sur la profondeur
 - Les nœuds de profondeur l n'ont pas de successeurs
- Exemple pour $l = 2$



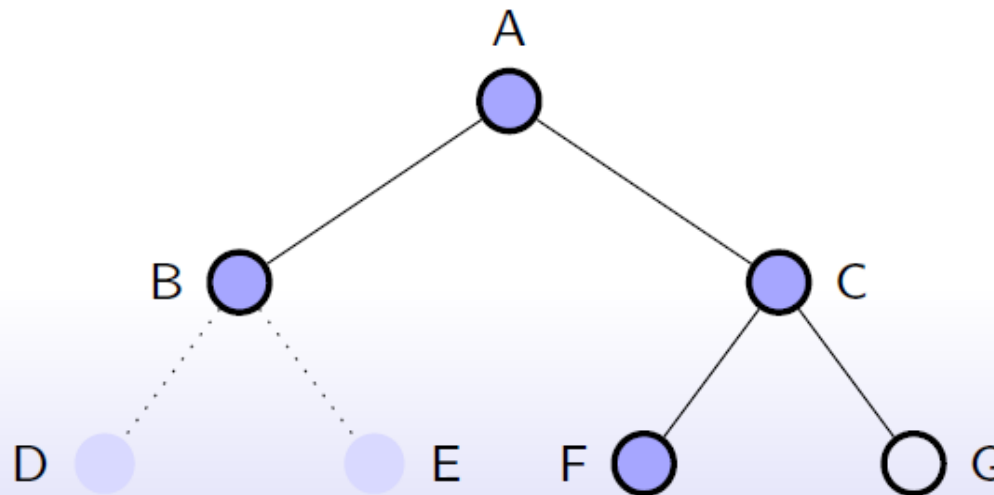
4- Recherche en profondeur limitée

- Algorithme de recherche en profondeur d'abord, mais avec une limite l sur la profondeur
 - Les nœuds de profondeur l n'ont pas de successeurs
- Exemple pour $l = 2$



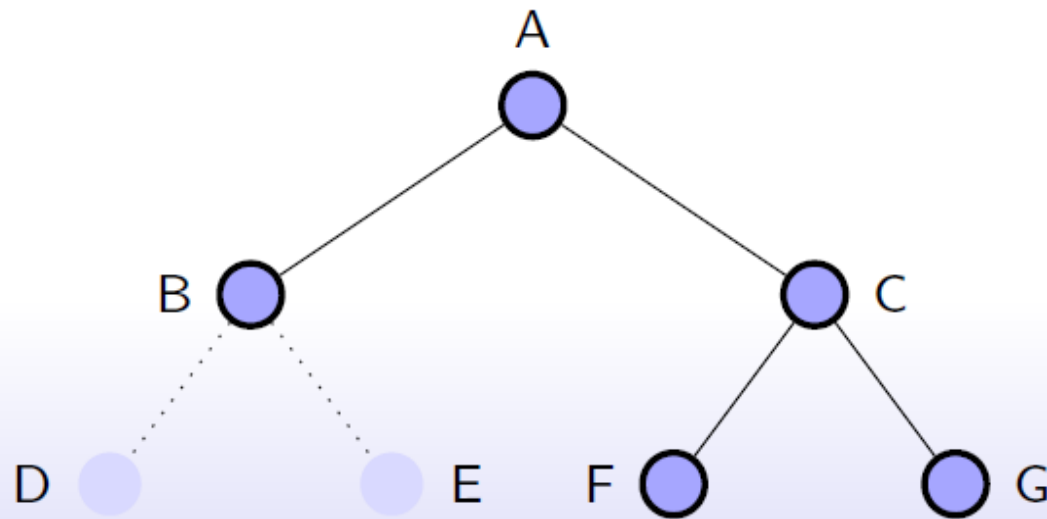
4- Recherche en profondeur limitée

- Algorithme de recherche en profondeur d'abord, mais avec une limite l sur la profondeur
 - Les nœuds de profondeur l n'ont pas de successeurs
- Exemple pour $l = 2$



4- Recherche en profondeur limitée

- Algorithme de recherche en profondeur d'abord, mais avec une limite l sur la profondeur
 - Les nœuds de profondeur l n'ont pas de successeurs
- Exemple pour $l = 2$



4- Recherche en profondeur limité

- Complet si $l \geq d$
- Complexité en temps : $O(b^l)$
- Complexité en espace : $O(bl)$
- Non optimale

5- Recherche en profondeur itérative

- Profondeur limitée, mais en essayant toutes les profondeurs: 0, 1, 2, 3, ...
- Evite le problème de trouver une limite pour la recherche profondeur limitée
- Combine les avantages de largeur d'abord (complète et optimale), mais a la complexité en espace de profondeur d'abord

5- Recherche en profondeur itérative

```
function ITERATIVE-DEEPENING-SEARCH(problem) returns a solution, or fail-  
ure  
  inputs: problem, a problem  
  for depth  $\leftarrow$  0 to  $\infty$  do  
    result  $\leftarrow$  DEPTH-LIMITED-SEARCH(problem, depth)  
    if result  $\neq$  cutoff then return result
```

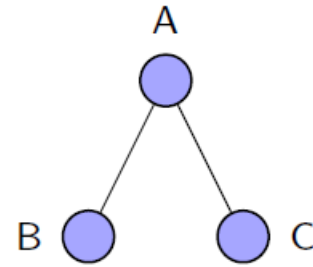
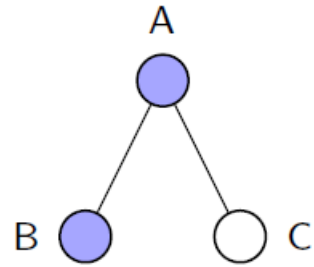
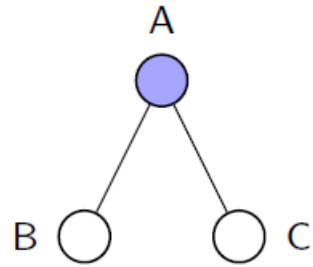
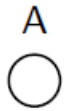
5- Recherche en profondeur iterative $l = 0$



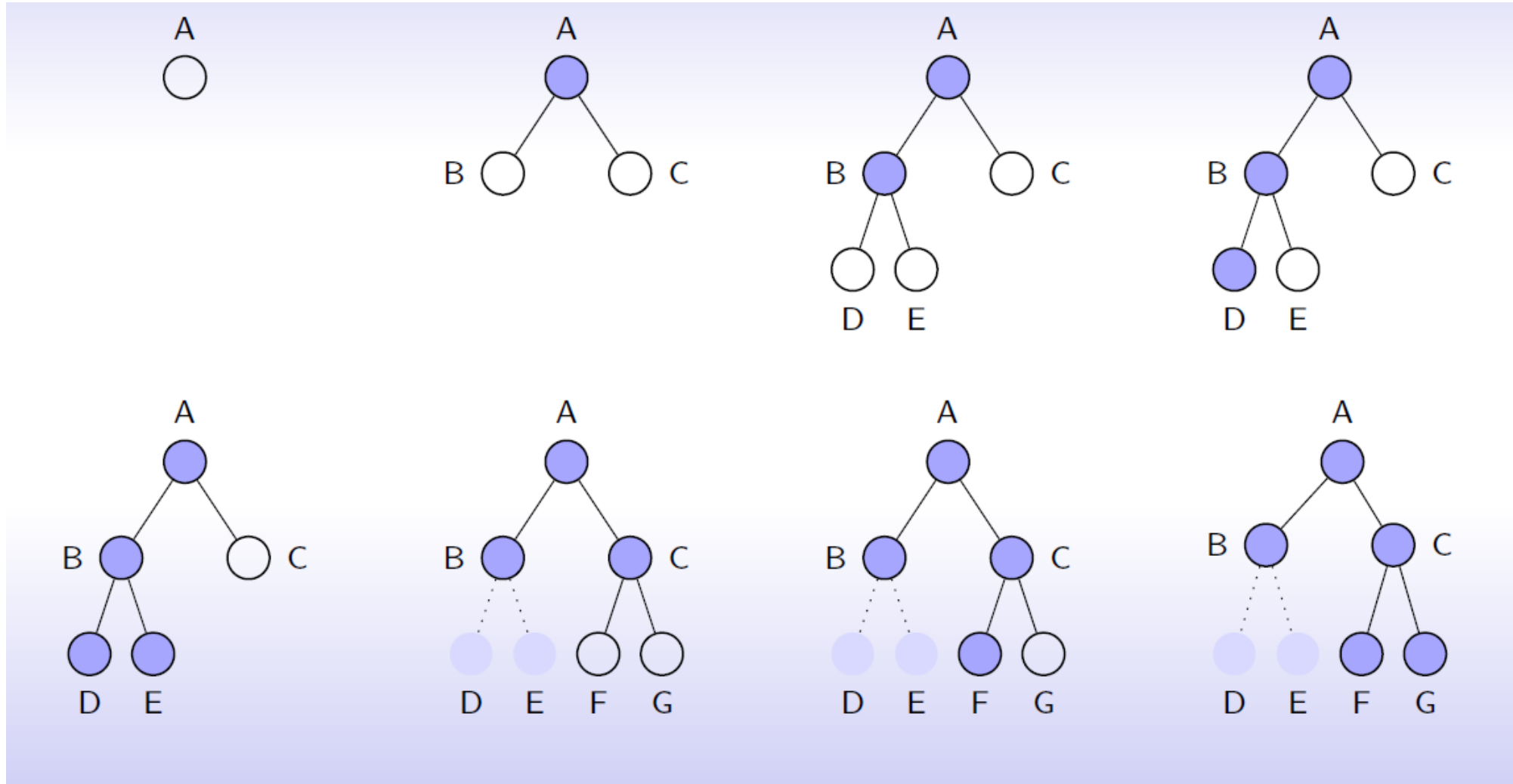
A
○

A
●

5- Recherche en profondeur iterative $l = 1$



5- Recherche en profondeur iterative $l = 2$



5- Recherche en profondeur iterative

- Complet
- Complexité en temps :

$$(d + 1)b^0 + db^1 + (d - 1)b^2 + \dots + b^d = O(b^d)$$

- Complexité en espace : $O(bd)$
- Optimale : oui, si le coût de chaque action est de 1. Peut être modifiée pour une stratégie de coût uniforme

Comparaison des algorithmes de recherche aveugles

- Comparaison de 6 algorithmes selon les 4 critères d'évaluation retenus

avec: b = facteur de branchement

d = profondeur de la solution

m = profondeur maximum de l'arbre de recherche

l = limite de la profondeur de recherche

Criterion	Breadth-First	Uniform-Cost	Depth-First	Depth-Limited	Iterative Deepening	I (i)
Time	b^d	b^d	b^m	b^l	b^d	
Space	b^d	b^d	bm	bl	bd	
Optimal?	Yes	Yes	No	No	Yes	
Complete?	Yes	Yes	No	Yes, if $l \geq d$	Yes	

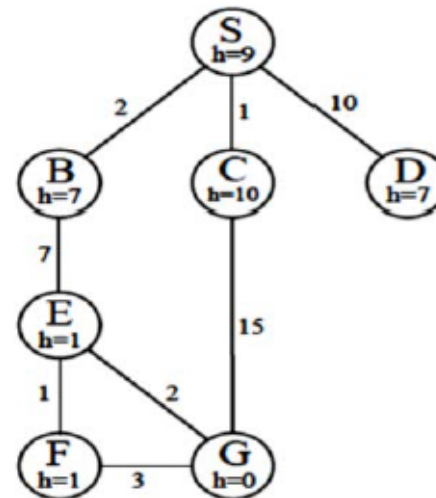
Recherche dans les Graphes

- **2 méthodes:**
 - **Méthode 1:** Faire la même chose que dans un arbre, mais garder une liste des noeuds déjà visités qui ne doivent pas être re-visités. → **On peut s'y perdre!!!**
 - **Méthode 2:** Retracer le graphe en un arbre en suivant tous les chemins possibles jusqu'à ce qu'ils ne puissent plus être allongés sans créer de boucle.

Exercice 1

Considérez le graphique de recherche présenté à droite. S est l'état de départ et G est l'état de but. Toutes les arêtes sont bidirectionnelles. Pour chacune des stratégies de recherche suivantes, indiquez le chemin, son coût et complexités en temps et espace. Donner le calcul intermédiaire. (En cas d'égalité, utiliser l'ordre alphabétique)

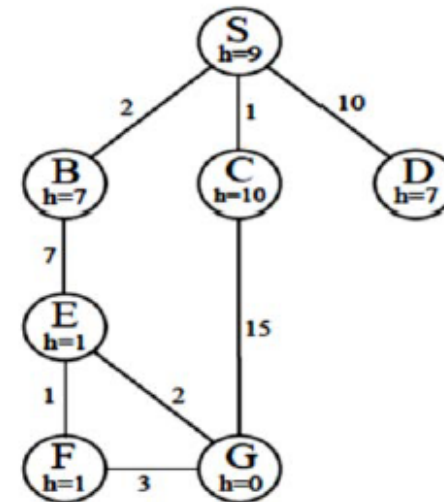
- a) Profondeur
- b) Largeur
- c) Cout Uniforme
- d) Gourmande
- d) A*



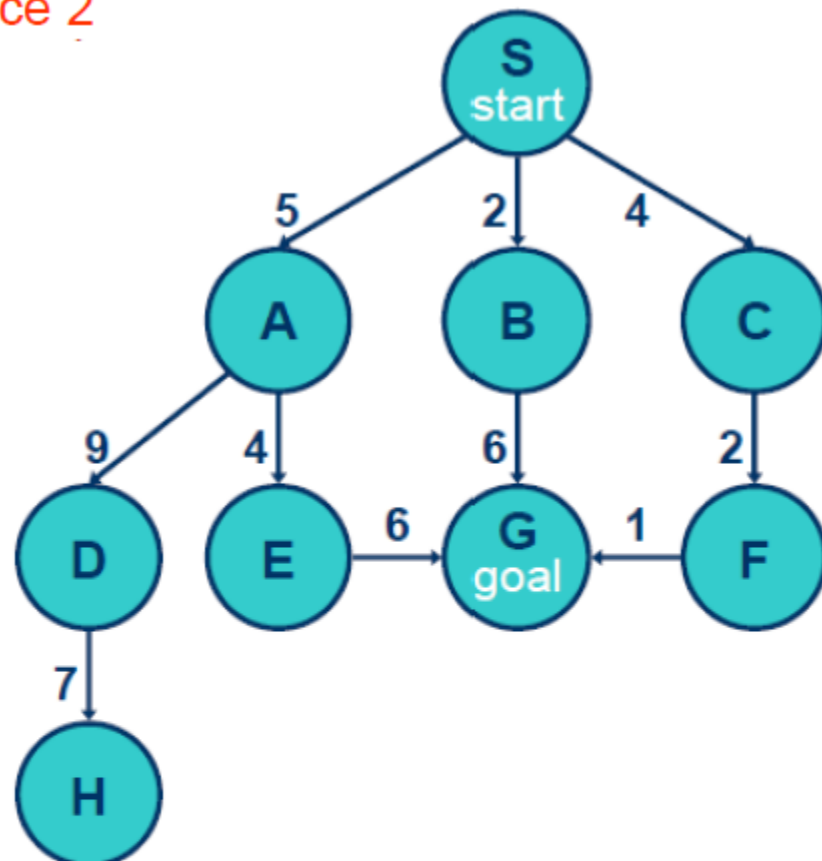
Exercice 1

Considérez le graphique de recherche présenté à droite. S est l'état de départ et G est l'état de but. Toutes les arêtes sont bidirectionnelles. Pour chacune des stratégies de recherche suivantes, indiquez le chemin, son coût et complexités en temps et espace. Donner le calcul intermédiaire. (En cas d'égalité, utiliser l'ordre alphabétique)

- a) Profondeur **S-B-E-F-G**
- b) Largeur **S-C-G**
- c) Cout Uniforme **S-B-E-G**
- d) Gourmande **S-B-E-G**
- d) A* **S-B-E-G**



Exercice 2

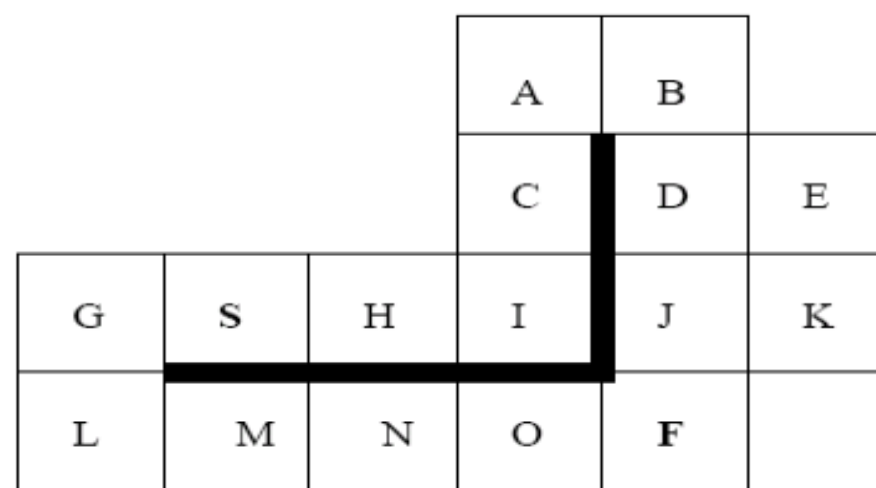


Dans quel ordre les nœuds sont développés pour chacun des algorithmes?

- Largeur d'abord
- Coût uniforme
- Profondeur d'abord
- Profondeur limitée
- Profondeur itérative

Exercice 3

Considérez le plan suivant où les successeurs de chaque case sont les cases adjacentes dans les directions Nord, Est, Sud et Ouest sauf à la limite du plan ou s'il y a une barrière (ligne épaisse). Par exemple $\text{successeurs}(J) = \{K, F, D\}$. On suppose que chaque opérateur (Nord, Sud, Est, Ouest) à coût = 1.



Le problème est de trouver un chemin de S vers F. On suppose que les successeurs sont engendrés dans l'ordre Est, Sud, Ouest et Nord.

Donnez l'ordre des noeuds développés pour les 4 méthodes de recherche suivante:

1. Recherche en largeur d'abord, Coût Uniforme profondeur d'abord, et Profondeur d'approfondissement itératif.

On suppose qu'on détecte des cycles, c.-à-d. on ne développe pas un noeud déjà développé avant.

FIN